Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №7
по курсу «Технологии глубоких нейронных сетей»
на тему «Определение изображений»

Выполнил:
студент группы 23ВВИ2
Юров Д.М.

Принял:
Митрохин М.А.

Пенза 2026

Цель работы: изучить возможности библиотек UltraLytics, OpenCV и предобученной модели YOLOv8 для решения задач обнаружения объектов на изображениях, создать и разметить собственный набор данных, обучить на нём нейронную сеть.

Задание:

1) Активировать среду выполнения dnnenv в командной строке cmd или Anaconda Prompt;
2) Установить пакеты OpenCV и UltraLytics;
3) Проверить корректность установки пакетов, запустив тестовый скрипт;
4) Запустить IDE, открыть файл Lab7_yolo_detector.py, изучить комментарии в программе и выполнить предложенный пример кода;
5) После ознакомления с примером выполнить основное задание:
   - создать собственный набор данных для обнаружения объектов;
   - выполнить разметку изображений в системе CVAT;
   - подготовить датасет в формате YOLO;
   - загрузить предобученную нейронную сеть из библиотеки Ultralytics;
   - обучить модель на созданном наборе данных;
   - проверить качество обнаружения объектов на тестовых или валидационных изображениях.

При создании датасета допускается:

- самостоятельно фотографировать изображения;
- скачивать изображения из Интернета вручную.

Использование готовых существующих датасетов из Интернета не допускается.

Ход выполнения:

1) Установка пакетов OpenCV и UltraLytics

Запустил Anaconda Prompt, активировал среду «dnnenv» и выполнил команду «conda install opencv» (рис. 1)

```
(base) C:\Users\izada>activate dnnenv

(dnnenv) C:\Users\izada>conda install opencv
3 channel Terms of Service accepted
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

Рисунок 1 - установка OpenCV 1

Пакет OpenCV успешно установлен (рис. 2).

```
Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done

(dnnenv) C:\Users\izada>
```

Рисунок 2 - установка OpenCV 2

Выполнил команду «pip install ultralytics» (рис. 3).

```
(dnnenv) C:\Users\izada>pip install ultralytics
Collecting ultralytics
  Downloading ultralytics-8.4.24-py3-none-any.whl.metadata (39 kB)
```

Рисунок 3 - установка UltraLytics 1

Пакет UltraLytics успешно установлен (рис. 4).

```
Downloading ultralytics-8.4.24-py3-none-any.whl (1.2 MB)
   ---------------------------------------- 1.2/1.2 MB 5.1 MB/s  0:00:00
Downloading polars-1.39.3-py3-none-any.whl (823 kB)
   ---------------------------------------- 824.0/824.0 kB 8.9 MB/s  0:00:00
Downloading polars_runtime_32-1.39.3-cp310-abi3-win_amd64.whl (47.0 MB)
   ---------------------------------------- 47.0/47.0 MB 26.2 MB/s  0:00:01
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: polars-runtime-32, polars, ultralytics-thop, ultralytics
Successfully installed polars-1.39.3 polars-runtime-32-1.39.3 ultralytics-8.4.24 ultralytics-thop-2.0.18

(dnnenv) C:\Users\izada>
```

Рисунок 4 - установка UltraLytics 2

После установки запустил на выполнение тестовый скрипт командой «python lead_test.py», в консоли появилось «OK!» (рис. 5).

```
━━ 21.5MB 6.4MB/s 3.4s
[ WARN:0@9.281] global cap.cpp:177 cv::VideoCapture::open VIDEOIO(CV_IMAGES):

OpenCV(4.12.0) C:\miniconda3\conda-bld\opencv-suite_1764690171940\work\modules
215:Assertion failed) !_filename.empty() in function 'cv::CvCapture_Images::op



OK!

(dnnenv) D:\PSU\4_term\TGNS\res\lab7>
```

Рисунок 5 - выполнение тестового скрипта

2) Ознакомление с примером

С помощью Anaconda Prompt запустил Spyder, прочитал комментарии и начал поэтапно выполнять код. Вывел результат обработки изображения-примера предобученной моделью (рис. 6-7).

```
   ...: # Выводим результаты на экран при помощи OpenCV
   ...: cv2.imshow("YOLOv8", result.plot())
   ...: cv2.waitKey(0)
   ...: cv2.destroyAllWindows()
Ultralytics 8.4.24  Python-3.10.20 torch-2.10.0+cu126 CUDA:0 (NVIDIA GeForce RTX 2060, 6144MiB)
Setup complete  (12 CPUs, 63.8 GB RAM, 318.1/607.6 GB disk)

image 1/1 D:\PSU\4_term\TGNS\res\lab7\zadanie\image_1.jpg: 448x640 10 persons, 2 benchs, 1 bottle, 13.1ms
Speed: 1.4ms preprocess, 13.1ms inference, 1.7ms postprocess per image at shape (1, 3, 448, 640)
```

Рисунок 6 - код вывода результатов

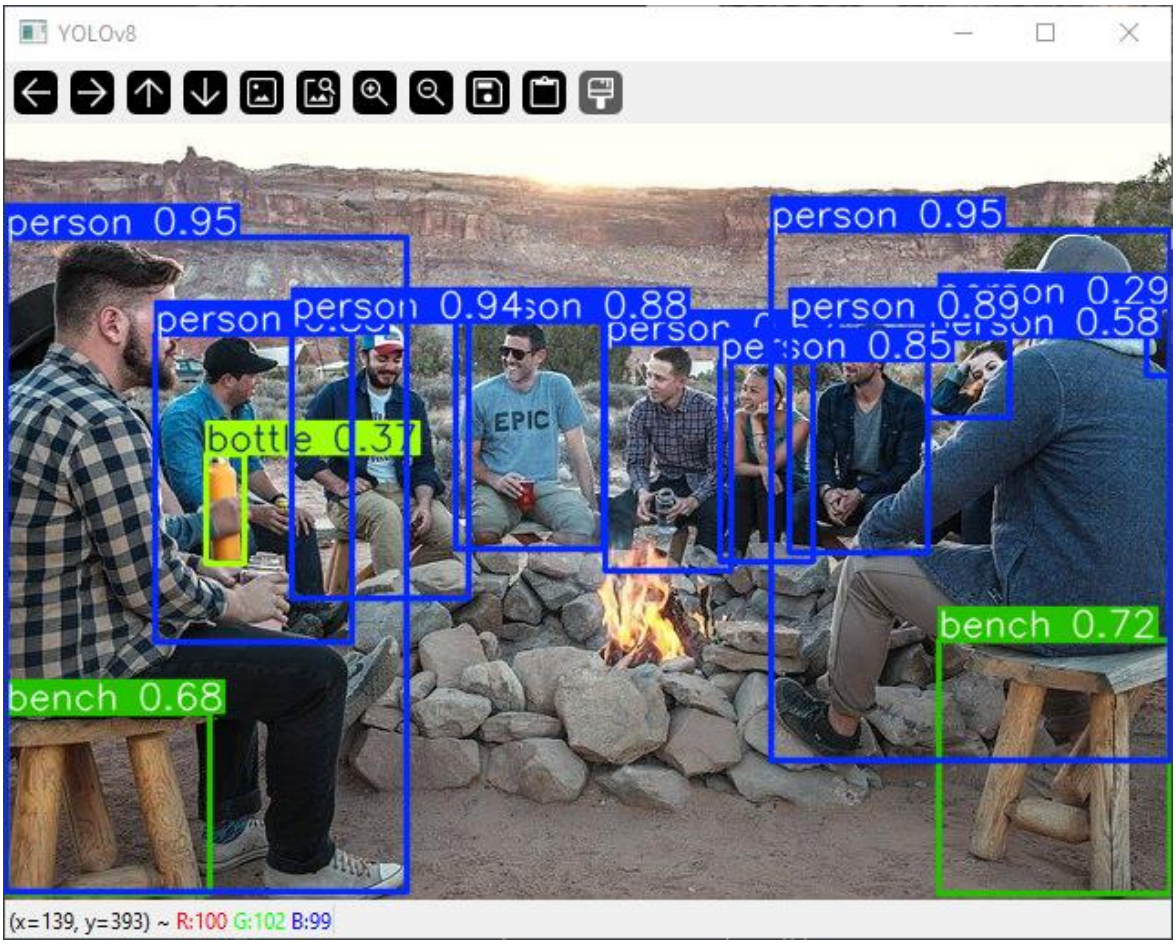

Рисунок 7 - плот результатов

Вывел результирующий плот работы функции отрисовки полей (рис. 8-9).

```
In [6]: annotated_img = draw_bboxes(img.copy(), results)
   ...: cv2.imshow("YOLOv8", annotated_img)
   ...: cv2.waitKey(0)
   ...: cv2.destroyAllWindows()
```

Рисунок 8 - код вывода результата

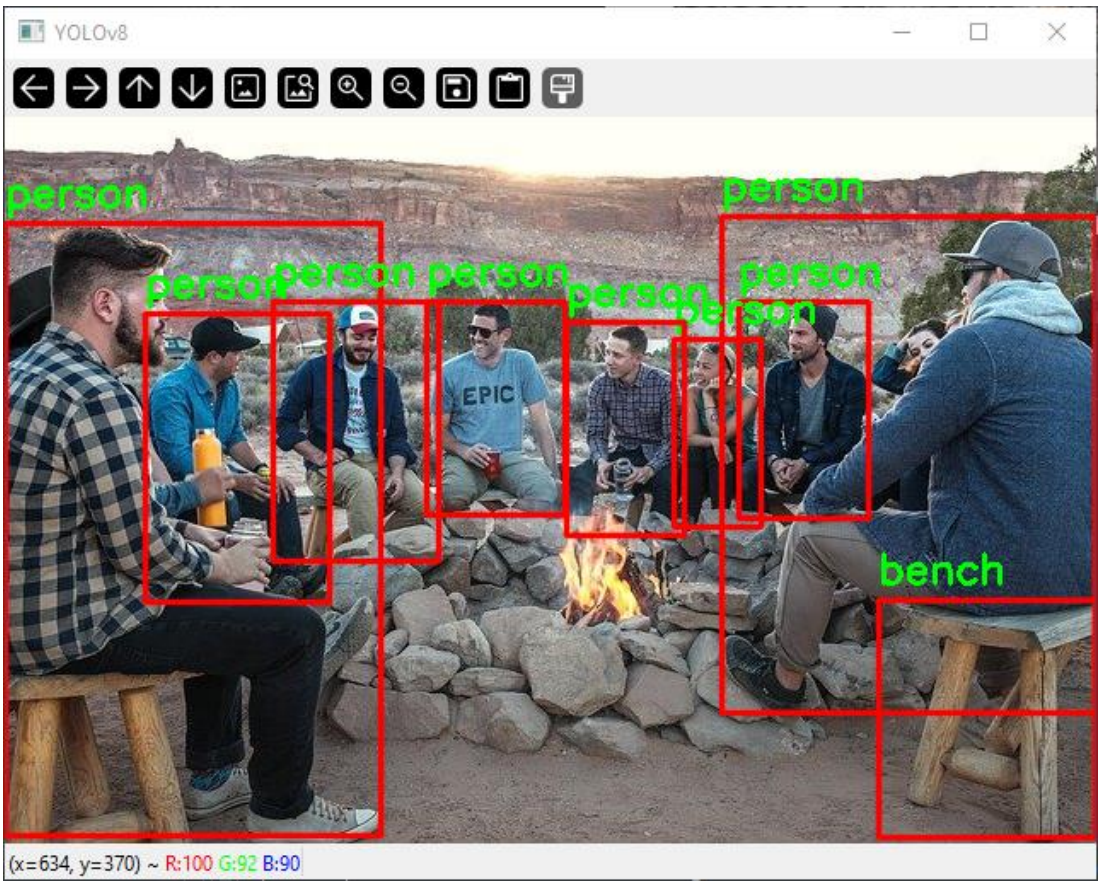

Рисунок 9 - вывод результирующего плота

Загрузил архив с датасетом по ссылке из комментария и распаковал его (рис. 10).

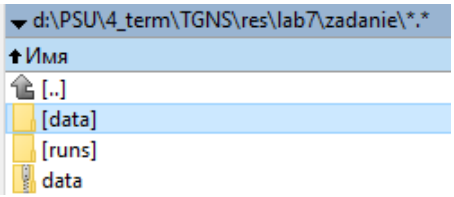

Рисунок 10 - загруженный датасет

Изменил путь к файлам датасета в файле «masked.yaml» (рис. 11).

```
masked.yaml U
D: > PSU > 4_term > TGNS > res > lab7 > zadanie > masked.yaml
path: data  # dataset root dir
train: D:\PSU\4_term\TGNS\res\lab7\zadanie\data\data\images\train # train images (relative to 'path')
val: d:\PSU\4_term\TGNS\res\lab7\zadanie\data\data\images\val # val images (relative to 'path')

# Classes
names:
  0: mask
  1: NOmask
```

Рисунок 11 - правка "masked.yaml"

Обучил нейронную сеть на загруженном датасете (рис. 12).

```
Starting training for 1 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
        1/1      2.18G      1.482      1.528      1.259         35        640: 100% ━━━━━━━━━━━━ 105/105 5.5it/s 19.2s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 4.9it/s 0.2s
                   all         13         47      0.546      0.905      0.637      0.397

1 epochs completed in 0.006 hours.
Optimizer stripped from D:\PSU\4_term\TGNS\res\lab7\zadanie\runs\detect\masks\train2\weights\last.pt, 22.5MB
Optimizer stripped from D:\PSU\4_term\TGNS\res\lab7\zadanie\runs\detect\masks\train2\weights\best.pt, 22.5MB

Validating D:\PSU\4_term\TGNS\res\lab7\zadanie\runs\detect\masks\train2\weights\best.pt...
Ultralytics 8.4.24  Python-3.10.20 torch-2.10.0+cu126 CUDA:0 (NVIDIA GeForce RTX 2060, 6144MiB)
Model summary (fused): 73 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 3.8it/s 0.3s
                   all         13         47      0.548      0.905      0.635      0.396
                  mask         13         37      0.354      0.811      0.378       0.21
                NOmask          4         10      0.743          1      0.891      0.583
Speed: 0.3ms preprocess, 14.3ms inference, 0.0ms loss, 2.2ms postprocess per image
Results saved to D:\PSU\4_term\TGNS\res\lab7\zadanie\runs\detect\masks\train2

image 1/1 D:\PSU\4_term\TGNS\res\lab7\zadanie\maksssksksss0.JPG: 480x640 1 mask, 2 NOmasks, 41.7ms
Speed: 3.3ms preprocess, 41.7ms inference, 1.9ms postprocess per image at shape (1, 3, 480, 640)

In [9]:
```

Рисунок 12 - обучение нейронной сети на загруженном датасете

Проверил работу обученной нейронной сети на видео-примере (рис. 13-14).

Рисунок 13 - проверка работоспособности сети 1

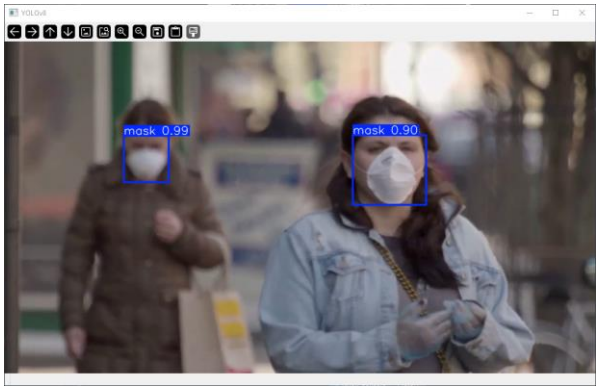

Рисунок 14 - проверка работоспособности сети 2

3) Создание собственного датасета

Загрузил из интернета («Яндекс.Картинки») 92 изображения людей, телефонов и людей с телефонами (рис. 15).

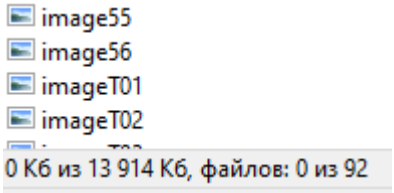

Рисунок 15 - загруженные изображения

Зашел на сайт CVAT и авторизировался с помощью Google. Создал новый проект «Lab7CVAT», в котором обозначил 2 класса: «human» и «Phone» (рис. 16).

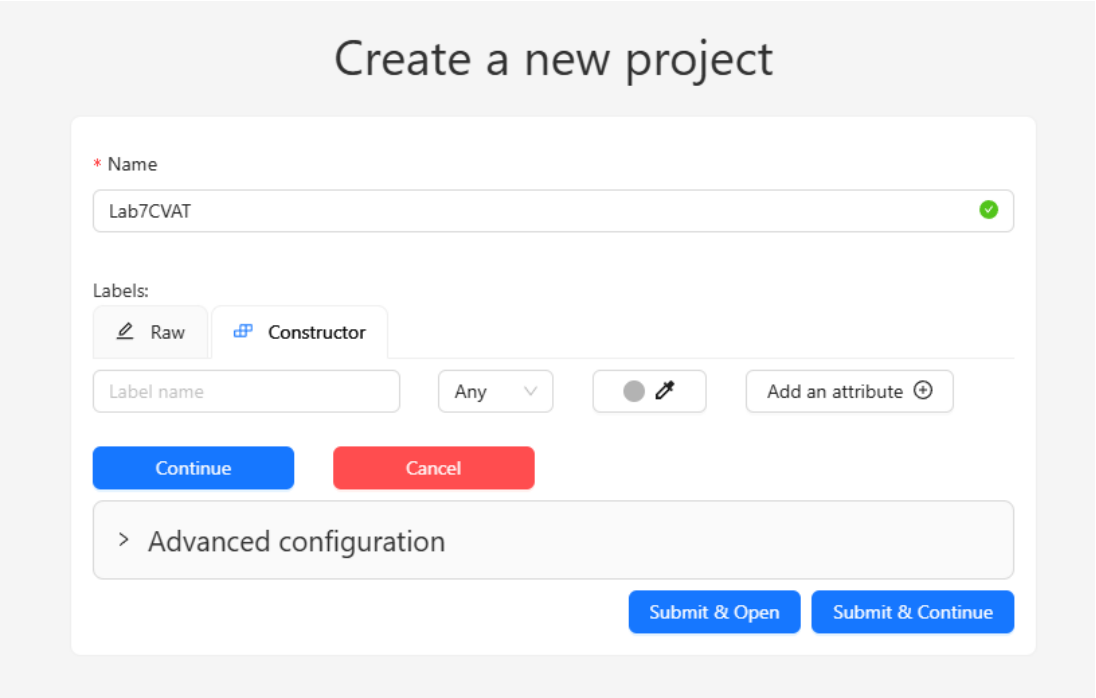

Рисунок 16 - создание нового проекта

Просмотрел параметры проекта и назначил себя ответственным за него (рис. 17).

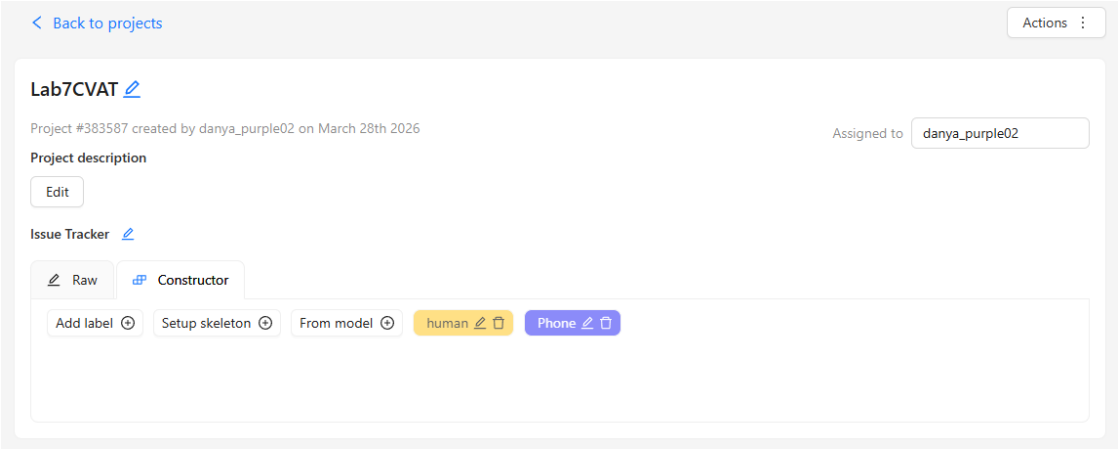

Рисунок 17 - параметры проекта

Создал задачу для разметки тренировочного сета (рис. 18).

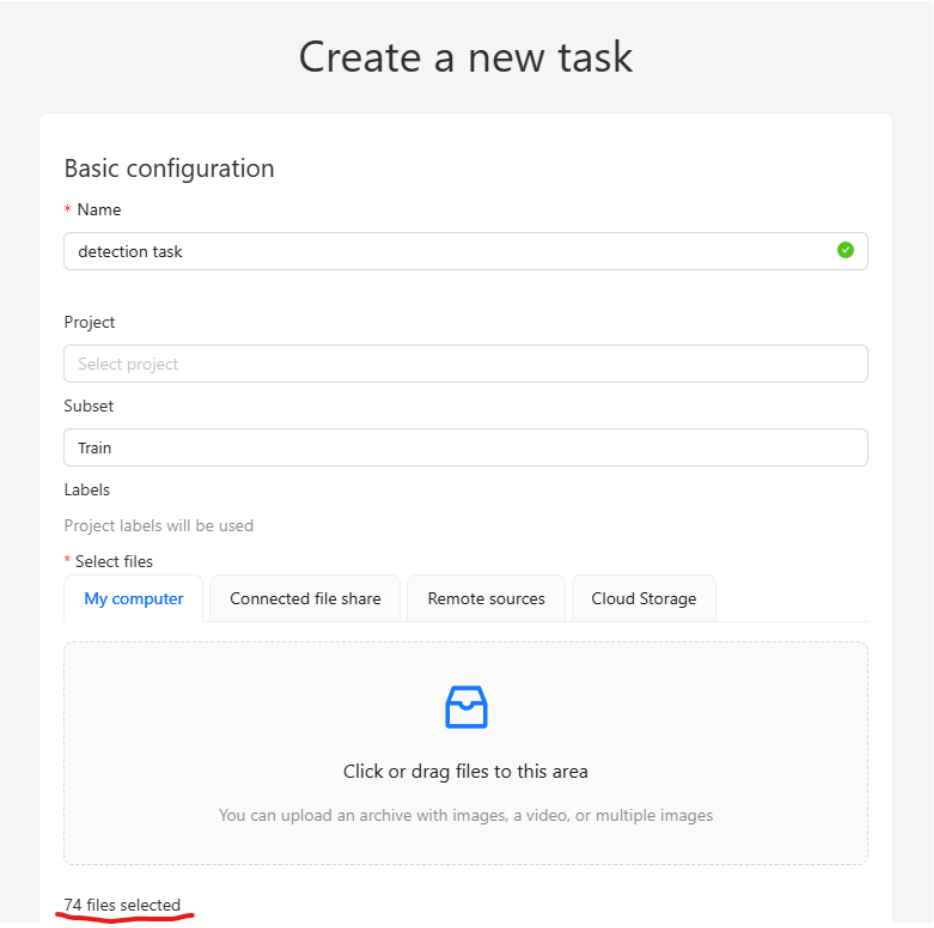

Рисунок 18 - создание задачи

Перешел в задачу, назначил ответственного (рис. 19).

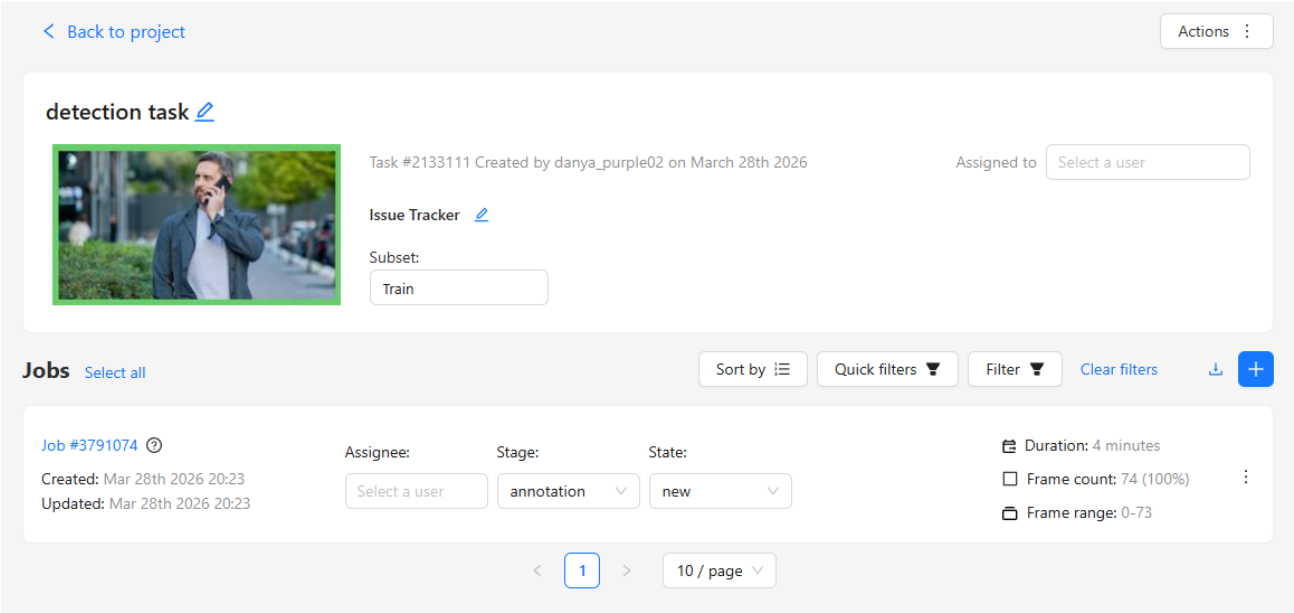

Рисунок 19 - параметры задачи

Открыл автоматически созданную работу (рис. 20) и приступил к разметке (рис. 21-22).

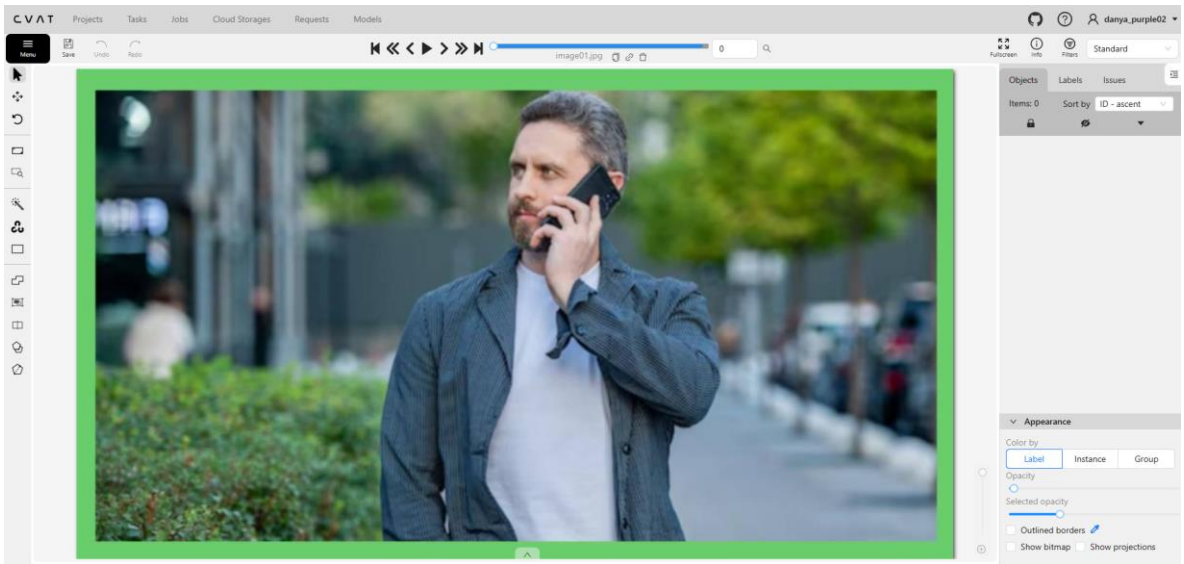

Рисунок 20 - интерфейс работы

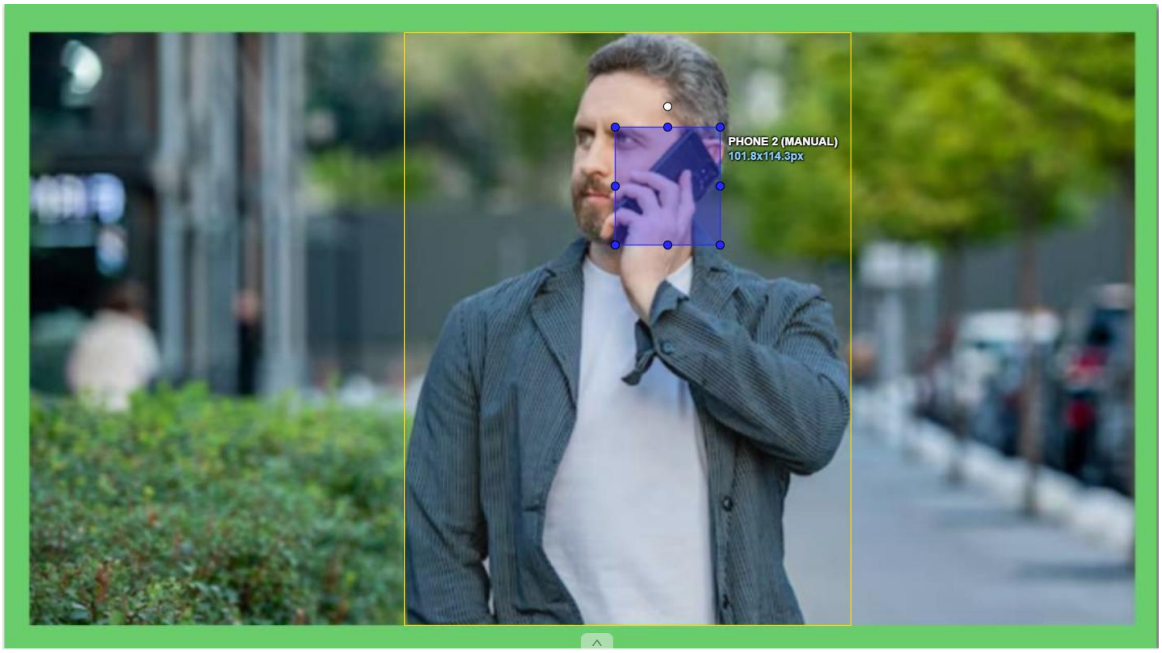

Рисунок 21 - ручная разметка объектов 1

Рисунок 22 - ручная разметка объектов 2

По завершению разметки экспортировал работу в датасет формата «YOLO 1.1» с изображениями (рис. 23), но бесплатный аккаунт не дал этого сделать (рис. 24), поэтому пришлось экспортировать без рисунков и структурировать датасет самостоятельно.

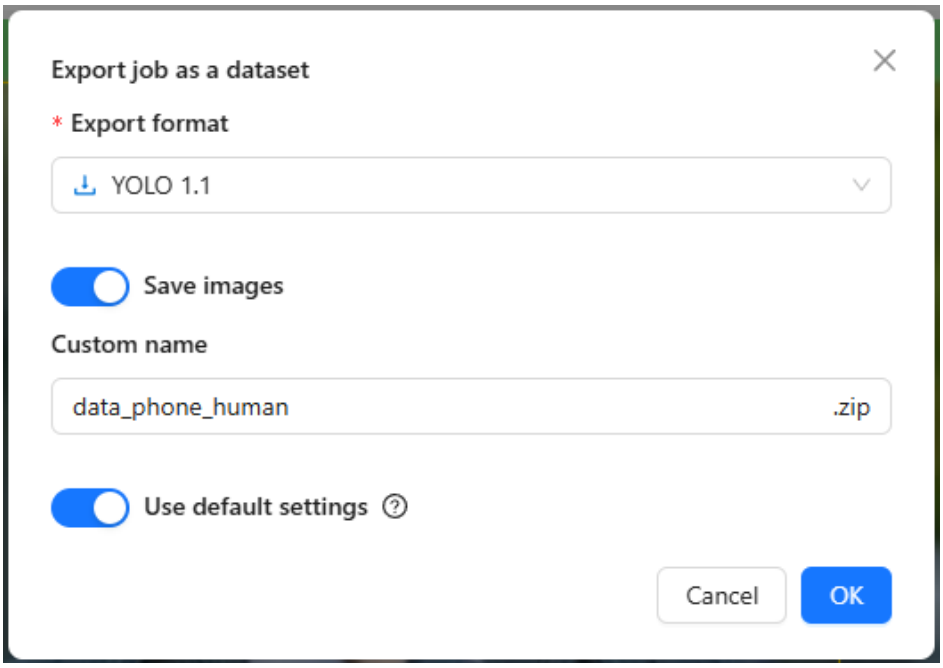

Рисунок 23 - экспорт разметки в датасет

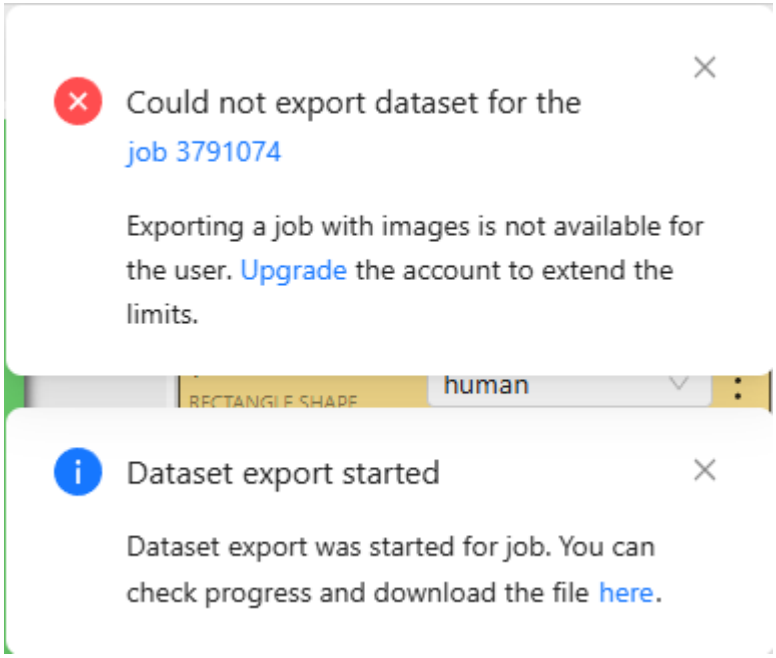

Рисунок 24 - ограничение бесплатного аккаунта

Структура датасета приложена на рисунках 25-26. Сет разделен на валидационный набор и тренировочный набор.

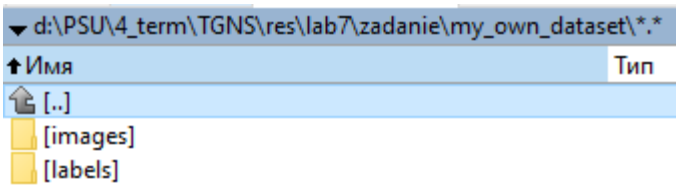

Рисунок 25 - структура сета 1

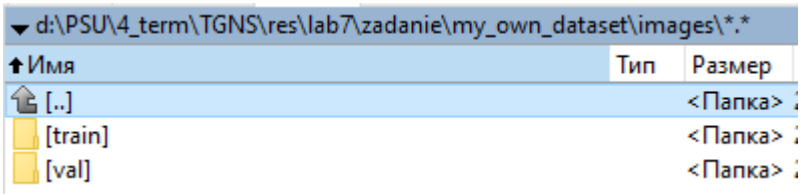

Рисунок 26 - структура сета 2

Создал файл «phone_human.yaml», в котором описал структуру датасета и классы (рис. 27).

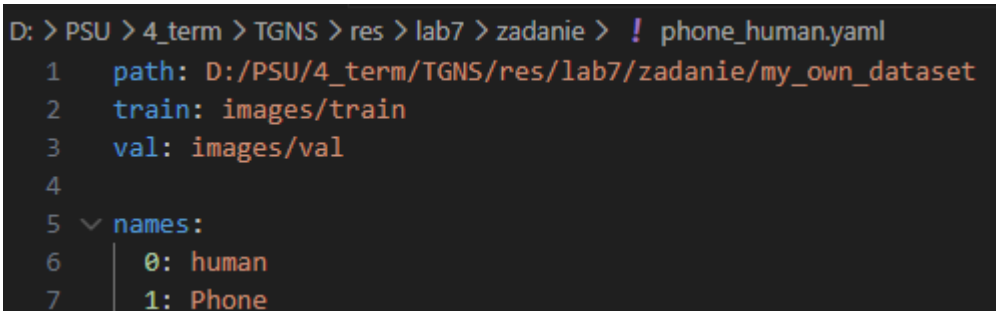

Рисунок 27 - файл "phone_human.yaml"

4) Обучение нейросети и решение поставленной задачи

Модифицировал код из примера, заменив ссылки на файлы и вырезав часть кода с определением объектов на видео.

Запустил проверку предобученной модели на изображении из своего датасета и вывел результат определения (рис. 28).

Рисунок 28 - результат определения

Запустил обучение сети на своем датасете (рис. 29) – сеть успешно обучилась (рис. 30).

```
model = YOLO("yolov8s.pt")
results = model.train(data="phone_human.yaml", epochs=20, batch=8,
                      project='human_phone', val=True, verbose=True)
```

Рисунок 29 - обучение сети на собственном датасете

```
Model summary (fused): 73 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 2/2 9.0it/s 0.2s
                   all         18        125      0.838       0.76      0.861      0.654
                 human         11         48       0.84      0.792      0.884      0.659
                 Phone         15         77      0.836      0.728      0.838      0.649
Speed: 0.6ms preprocess, 4.1ms inference, 0.0ms loss, 3.2ms postprocess per image
Results saved to D:\PSU\4_term\TGNS\res\lab7\zadanie\runs\detect\human_phone\train2
```

Рисунок 30 - результат обучения

После обучения вывел (рис. 31) несколько результатов проверки определения (рис. 32-)

```
In [11]: results = model("D:/PSU/4_term/TGNS/res/lab7/zadanie/my_own_dataset/images/
val/image54.jpg")
    ...:
    ...: # посмотрим что получилось
    ...: result = results[0]
    ...: # cv2.imshow("YOLOv8", result.plot())
    ...: # cv2.waitKey(0)
    ...: # cv2.destroyAllWindows()
    ...: plt.imshow(result.plot()[:, :, ::-1])
    ...: plt.show()

image 1/1 D:\PSU\4_term\TGNS\res\lab7\zadanie\my_own_dataset\images\val\image54.jpg:
320x640 11 humans, 46.9ms
Speed: 2.6ms preprocess, 46.9ms inference, 3.2ms postprocess per image at shape (1, 3,
320, 640)
```

Рисунок 31 - код вывода результата

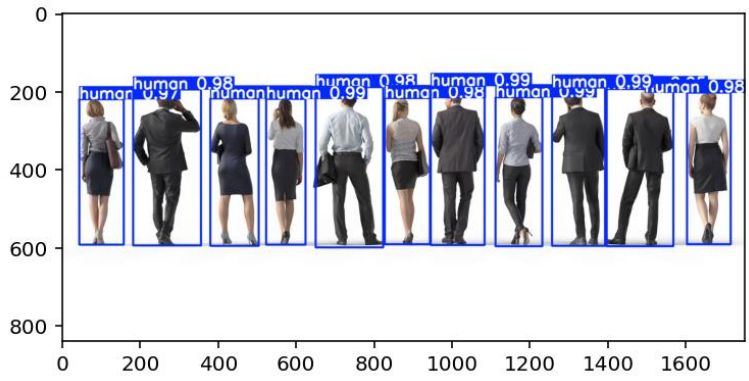

Рисунок 32 - результат 1

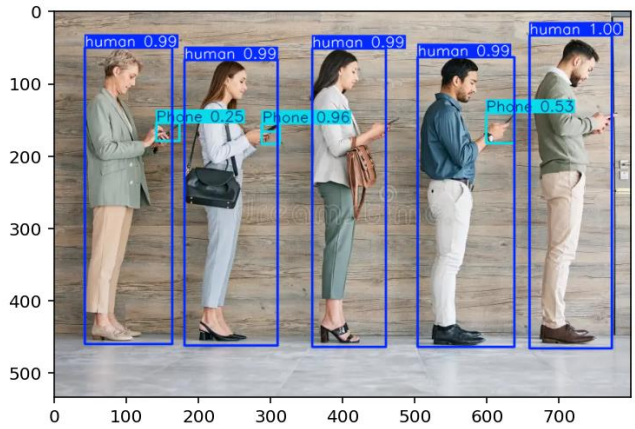

Рисунок 33 - результат 2

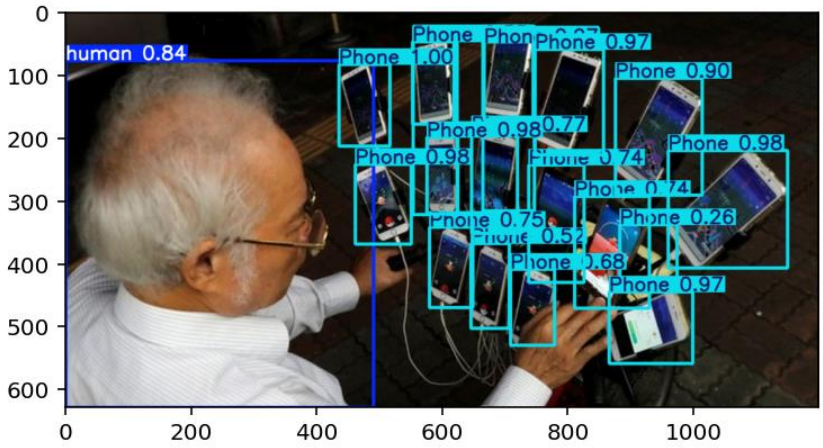

Рисунок 34 - результат 3

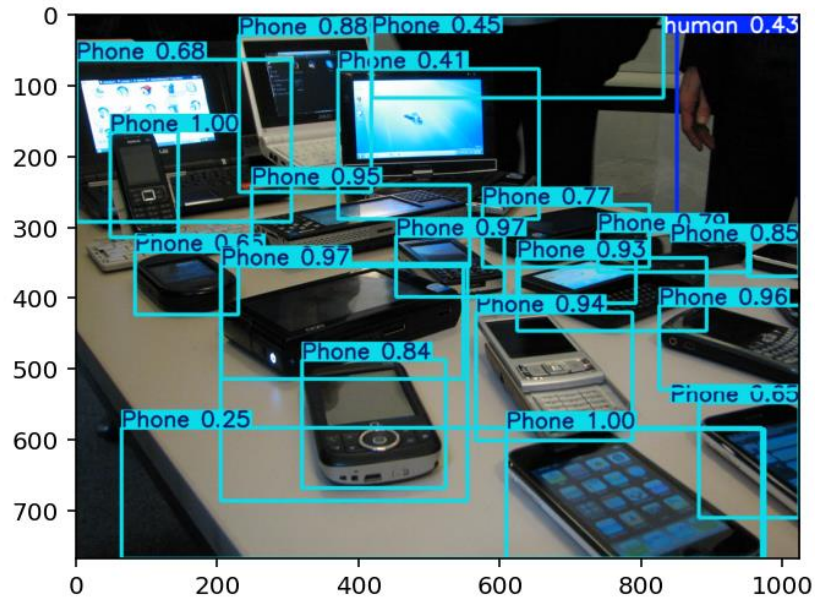

Рисунок 35 - результат 4

Вывод: в ходе выполнения лабораторной работы изучил возможности библиотек UltraLytics, OpenCV и предобученной модели YOLOv8 для решения задач обнаружения объектов на изображениях, создал и разметил собственный набор данных, обучил на нём нейронную сеть.

Репозиторий: https://github.com/danya-purple02/TGNS/tree/master/lab7